

TAM: Torque Adaptation Module for Robust Motion Transfer in Manipulation

Anonymous Author(s)

Affiliation

Address

email

Abstract: A policy tuned for one robot often behaves differently on another, whether due to the sim-to-real gap, unknown payloads, or the differing dynamics of two instances of the same robot. In contact-rich, dynamic manipulation, even small motion discrepancies can result in failure to track reference motion, since they disrupt the timing and modes of contact. Common remedies, such as domain randomization or system identification, either produce overly conservative task policies [1] or require data that must be recollected for each robot or payload [2]. We introduce the **Torque Adaptation Module (TAM)**, a learned module that adapts the torque commands sent to the robot to match the behavior of an ideal robot. TAM operates between the low-level controller that tracks the policy’s actions and the robot’s torque interface. It includes a history encoder that embeds proprioceptive history into a compact latent state and a torque adaptor that computes residual torque corrections. Because TAM depends only on proprioceptive history and not on policy observations, rewards, or the action space, the same TAM weights can be reused to adapt policies with different action spaces (joint targets, end-effector targets, or direct torques). The policies themselves do not need to be trained with domain randomization of robot parameters. Instead, we offload the need for domain randomization to TAM by training it entirely in randomized simulation, using multi-robot pretraining followed by a robot-specific fine-tuning step that still requires no real-robot data. We evaluate TAM zero-shot on a real Franka Panda robot across dynamic manipulation tasks with unknown payloads that include a vision-based box pushing policy (from RL), a flip policy (from BC), and an MPC ball-on-plate balancing controller. Our experiments show that TAM significantly improves zero-shot real-robot execution compared to online system identification and RMA baselines and enables robust dynamic manipulation performance.

Keywords: Sim-to-real transfer, robot manipulation

1 Introduction

It is well known that discrepancies between simulated and real versions of a manipulator, different payloads, and even slight differences between instances of the same manipulator can cause significant deviations in motion behavior, leading to task failure [3, 4, 5]. This paper proposes a method that enables robust motion transfer in this broader sense: transferring a policy trained or designed for one reference robot to a target robot whose dynamics differ because of the sim-to-real gap, an added payload, or instance-to-instance variation.

Existing techniques usually address this problem either by adapting the model of the manipulator (for example via system identification or neural actuator modelling [6, 7, 8]) or by making the policy robust to such discrepancies via domain randomization [9, 10, 11]. While system identification methods can learn an accurate model, they require continual model adaptation as the robot instance or payload changes. Domain randomization is not a panacea either, as it can result in overly conservative policies that are invariant to a broad parameter set [1].

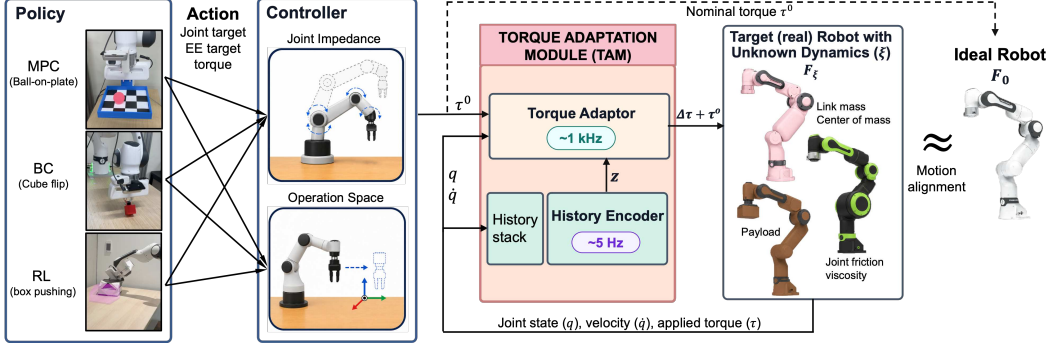


Figure 1: **TAM deployment interface.** Independently authored policies, such as MPC ball balancing, BC cube flipping, or RL box pushing, can be paired with any compatible low-level controller, which converts their actions, including joint targets, end-effector targets, or direct torque, into nominal torque τ_t^0 . TAM is inserted after this low-level-controller boundary and before the robot plant. The history encoder runs asynchronously at about 5 Hz, summarizing the online robot-side history into a latent z_t . The torque adaptor runs at the 1 kHz hardware control rate, reusing the most recent z_t to predict the residual torque $\Delta\tau_t$ at every control tick. The corrected torque $\tau_t^0 + \Delta\tau_t$ is applied to the real robot with unknown joint friction/viscosity, link-mass and center-of-mass mismatch, and payload, so the robot better follows the ideal-model response under the same τ^0 without changing the policy.

41 Methods such as UPOSI [12] and RMA [13, 14, 15] reduce some of the shortcomings of the ap-
 42 proaches above by learning to infer a latent representation of physical properties from observation
 43 history, and then conditioning the policy on the inferred dynamics. This approach is effective when
 44 the policy is trained jointly with the latent state and privileged information, however, it suffers from
 45 limited reuse: adding adaptation to a new Reinforcement Learning (RL), Behavior Cloning (BC), or
 46 Model Predictive Control (MPC) policy requires retraining or distilling the policy weights with the
 47 adaptation branch, which is costly or impossible for closed-source or third-party policies.

48 To overcome these limitations, we propose the **Torque Adaptation Module (TAM)**, a trajectory-
 49 conditioned torque adaptation layer. Our setting assumes an ideal robot model and a target robot.
 50 The ideal model is used to train or design a policy, while the target robot may have different physical
 51 parameters from the ideal model. At each timestep, the policy outputs an action a_t , and the corre-
 52 sponding low-level controller converts that action into the nominal torque τ_t^0 that would be applied
 53 to the ideal model. TAM observes the target robot’s joint-position, joint-velocity, and applied-torque
 54 history, then adds a residual torque to τ_t^0 so the target robot’s motion more closely matches the ideal-
 55 model motion under the same nominal torque.

56 **Contributions.** Our key contribution is to move the adaptation layer below both the policy and
 57 the low-level controller, to the torque interface level, as shown in Fig. 1. The first advantage of
 58 doing this is that TAM does not require access to policy observations, rewards, task labels, or policy
 59 internals. It only requires that the policy action can be converted to nominal torque. Policies can
 60 therefore be trained via RL or BC or computed via MPC on the ideal robot without robot-parameter
 61 randomization, while TAM handles robot-side mismatch during deployment. The second advantage
 62 is that it allows TAM to adapt nominal torques at 1 kHz, which enables fast, dynamic motion.

63 We evaluate TAM across real-world and simulated robust-transfer settings. The real-world experi-
 64 ments include tracking with unknown physical parameters and payload, vision-based RL box push-
 65 ing, a behavior-cloned flip policy, and an MPC ball-on-plate policy, alongside simulated cross-robot
 66 tracking across multiple manipulator models. Across these settings, TAM uses a single shared set
 67 of weights to enable real-world transfer of learned RL and BC policies without robot-side domain
 68 randomization, and of MPC plans optimized against the ideal model, outperforming online system
 69 identification and RMA baselines.

70 **Related Work.** Appendix A discusses how our work relates to existing literature on sim-to-real
 71 transfer, adaptive control, RMA, neural actuator models, policy adaptation, and residual learning.

2 Problem Definition and Scope

We study sim-to-real transfer for *black-box skills* (i.e., policies obtained via reinforcement learning (RL), behavior cloning (BC), or hand-designed via model predictive control (MPC)) deployed on robots whose dynamics differ from those assumed when the skill was trained. Let the robot have N actuated joints with joint positions, velocities, and accelerations $q_t, \dot{q}_t, \ddot{q}_t \in \mathbb{R}^N$. A skill is a policy π producing an action $a_t = \pi(o_t) \in \mathcal{A}$ from the current observation o_t , where the *action space* $\mathcal{A}$ may encode joint targets (with joint impedance), end-effector targets, resolved by operational-space control (OSC), or direct joint torques. A *low-level controller* $\mathcal{C} : \mathcal{A} \times \mathbb{R}^N \times \mathbb{R}^N \rightarrow \mathbb{R}^N$ converts the action together with the current proprioception into a *nominal torque* $\tau_t^0 = \mathcal{C}(a_t, q_t, \dot{q}_t)$.

Let $w_t \in \mathbb{R}^N$ denote an external generalized torque acting on the joints (e.g., a contact wrench mapped through $J^\top$), and let $F : \mathbb{R}^N \times \mathbb{R}^N \times \mathbb{R}^N \times \mathbb{R}^N \rightarrow \mathbb{R}^N$ denote a forward-dynamics map returning the realized acceleration $\ddot{q}_t = F(q_t, \dot{q}_t, \tau_t, w_t)$. The *ideal robot* has known deterministic dynamics F_0 , used as the reference model while obtaining π and $\mathcal{C}$ (e.g., URDF provided by the manufacturer along with rigid-body inertial parameters, no joint friction, etc.). The *target robot* has dynamics F_ξ , generally stochastic and parameterized by $\xi \in \Xi$ (friction, damping, inertial mismatch, payload, dead zones, actuator bias). On real hardware, ξ is a single fixed but unknown realization; in simulation, we assume that ξ is sampled from a distribution p_Ξ over Ξ during TAM training. Ideal and target robots share kinematic structure and actuator ordering, so the same τ_t^0 generally produces different accelerations on the two: $F_\xi(q_t, \dot{q}_t, \tau_t^0, w_t) \neq F_0(q_t, \dot{q}_t, \tau_t^0, w_t)$.

To recover the intended outcome on the target robot without retraining π , we insert a learned module at the *torque interface*, the boundary between $\mathcal{C}$ and the robot. The TAM module reads τ_t^0 and emits a *residual correction* $\Delta\tau_t \in \mathbb{R}^N$, yielding the *corrected torque* $\tau_t^c = \text{clip}(\tau_t^0 + \Delta\tau_t, \tau_{\min}, \tau_{\max})$ actually commanded to the robot, where $\tau_{\min}, \tau_{\max} \in \mathbb{R}^N$ are the target robot’s jointwise actuator limits. The *applied torque* recorded in the proprioceptive history is then $\tau_t^a = \tau_t^c$.

Problem statement. Learn a residual policy π_Δ for torque adaptation, which produces $\Delta\tau_t$ from signals available at deployment such that the target robot driven by τ_t^c locally matches the ideal robot driven by τ_t^0 : $F_\xi(q_t, \dot{q}_t, \tau_t^c, w_t) \approx F_0(q_t, \dot{q}_t, \tau_t^0, w_t)$. Note that w_t is not observed by π_Δ at deployment; it is treated as known only inside the training-time label generator.

To train π_Δ in simulation, we assume access to inverse torque-command maps Φ_0 and Φ_ξ : given a desired acceleration, $\Phi_\xi(q_t, \dot{q}_t, \ddot{q}_t, w_t)$ returns the command torque that produces $\ddot{q}_t$ under F_ξ and w_t , with Φ_0 defined analogously for F_0 . These maps are used only to synthesize ground-truth $\Delta\tau$ labels from simulator rollouts; their analytic form is given in Appendix C.1 and used in Section 3.1.

Scope and limits. The setup requires (i) access to the torque interface, (ii) access to Φ_ξ in simulation for label generation, and (iii) matched kinematic structure and actuator ordering between ideal and target robots. Hardware safety propagates through clipping: when alignment would require a torque outside $[\tau_{\min}, \tau_{\max}]$, τ_t^c saturates and the residual mismatch cannot be removed, so payloads and contact loads must remain within the actuator budget.

3 Torque Adaptation Module

3.1 Data Generation and Training Objective

Figure 2 summarizes the data-generation and training pipeline. As shown in the figure and caption, task-agnostic joint-motion rollouts produce robot-side histories, while analytic dynamics labels provide the torque target. The main learning signal is local torque matching: for a recorded state $(q_t, \dot{q}_t)$ and nominal torque query τ_t^0 , the ideal model defines the desired local response through $F_0(q_t, \dot{q}_t, \tau_t^0, w_t)$, and the teacher corrected torque is the actuator command that makes the perturbed model realize that response:

$$\tau_\xi^*(q_t, \dot{q}_t, \tau_t^0, w_t) = \Phi_\xi(q_t, \dot{q}_t, F_0(q_t, \dot{q}_t, \tau_t^0, w_t), w_t), \quad (1)$$

where Φ_ξ is the inverse torque-command map introduced in Section 2 and detailed in Appendix C.1.

Given the sampled nominal torque τ_t^0 , let $\hat{\tau}_{\theta,t}^c$ denote TAM’s corrected-torque prediction. The target corrected torque is $\tau_{\xi,t}^* = \tau_\xi^*(q_t, \dot{q}_t, \tau_t^0, w_t)$. The loss compares these two corrected torques, $\mathcal{L}_\tau = \rho_\delta(\hat{\tau}_{\theta,t}^c - \tau_{\xi,t}^*)$. Here $\rho_\delta(\cdot)$ is the elementwise Huber penalty on the joint-torque residual, averaged

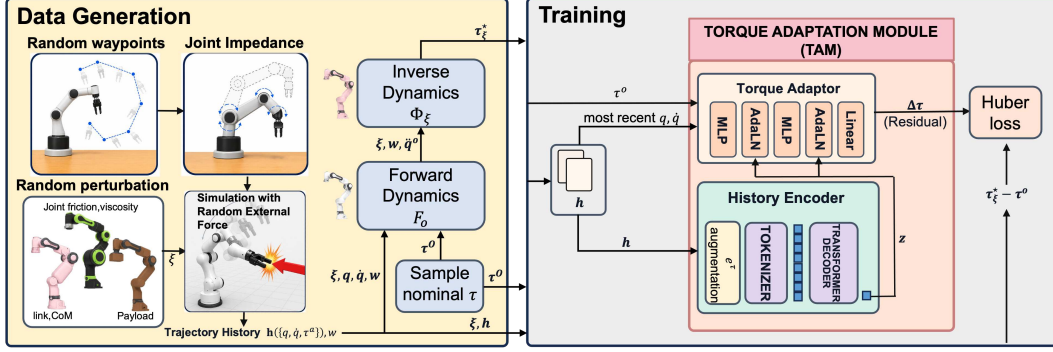


Figure 2: **TAM data generation and training.** Training data are task-agnostic free-space joint-motion rollouts, not downstream task demonstrations. For each rollout, we sample perturbation parameters ξ and synthetic randomized joint-space waypoint references designed to cover diverse positions, velocities, torques, and dynamics mismatches. A joint impedance low-level controller tracks these waypoints, producing histories $h_t = \{q, \dot{q}, \tau^a\}$, and we inject smooth external generalized-torque perturbations w_t to mimic the effect of object interaction. For each nominal torque τ_t^0 , the ideal forward dynamics F_0 provides the nominal acceleration, and the perturbed inverse command map Φ_ξ computes the teacher torque $\tau_{\xi,t}^*$ needed to realize that ideal response. The history encoder maps h_t to a latent z_t , the torque adaptor predicts $\Delta\tau_t$, and the Huber loss is applied to the teacher residual $\tau_{\xi,t}^* - \tau_t^0$.

over joints. Rollout generation, randomization ranges, nominal-torque query sampling, external-torque handling, the Huber threshold, closed-loop rollout loss, and training-time augmentations are detailed in Appendix B.

Although TAM is trained on randomized motion rollouts rather than downstream task demonstrations, its supervision is not tied to a particular task trajectory. TAM is trained as a local transition aligner: given $(q_t, \dot{q}_t)$, robot-side history h_t , and τ_t^0 , it predicts $\Delta\tau_t$ so the perturbed robot driven by τ_t^c matches the ideal response $F_0(q_t, \dot{q}_t, \tau_t^0, w_t)$. An RL pushing policy, BC flip policy, trajectory-tracking policy, and MPC ball-on-plate policy induce different long-horizon behaviors, but after their actions are converted to nominal torques, they all rely on the same local mapping from joint state and torque to next motion. Thus TAM requires coverage of relevant state, velocity, torque, and dynamics-mismatch regimes, not coverage of every downstream task trajectory.

3.2 Encoder-Torque Adaptor Architecture

The deployment interface in Figure 1 highlights the two runtime modules. The history encoder runs on long-horizon proprioceptive and applied-torque history to infer a compact latent state z that summarizes the current robot-side mismatch. The torque adaptor runs at the torque interface: it receives z , the current state history, and the nominal torque produced by the low-level controller, then outputs the residual torque used to form the corrected torque. This split lets TAM use long temporal context while keeping the control-rate correction module small enough to attach directly below different policies.

History encoder. The history encoder, shown in the TAM network block in Figure 2, summarizes how the target robot has been responding over a long-horizon torque history, which we use as a robustness choice. For simple tracking motions that keep the joints moving, a shorter window may be enough and would reduce computation. The longer window helps when the recent few seconds say little about the robot mismatch: a policy may hold almost constant commands, or object interaction may dominate the measured motion. In these cases, the encoder can still use earlier motion without contact together with the recent history. At each control step, it first stores a raw rolling buffer of measured position, measured velocity, and applied torque, $\{q_{t-i}, \dot{q}_{t-i}, \tau_{t-i}^a\}_{i=1}^K$. Here K is the number of controller-rate samples in the history buffer.

The raw buffer already contains the information needed to infer mismatch in principle, but it asks the network to discover inverse-dynamics structure from data alone. We therefore augment the history with a physics residual feature that answers a more direct question for each past sample: given the

measured motion, how much did the applied torque differ from the torque that the ideal model would have needed? This gives the encoder a past correction-like signal in torque units, which can help infer what residual torque should be applied next. We define this feature as $e_t^\tau = \tau_t^a - \Phi_0(q_t, \dot{q}_t, \ddot{q}_t, \mathbf{0})$. Here $\ddot{q}_t$ is the acceleration estimated from local position and velocity traces, and the last argument $\mathbf{0} \in \mathbb{R}^N$ sets the external generalized torque input of the ideal inverse torque-command map to zero. The encoder input then becomes the augmented rolling history $h_t = \{q_{t-i}, \dot{q}_{t-i}, \tau_{t-i}^a, e_{t-i}^\tau\}_{i=1}^K$. We next introduce the neural architecture used to process this long history. A naive causal Transformer [16] over the raw controller-rate sequence would see about 12,000 time samples per joint for the 12 s, 1 kHz history, making attention memory and computation unnecessarily expensive. Instead, the encoder follows patch-wise time-series tokenization [17]: it splits each joint history into overlapping temporal patches, embeds each patch as a token, and processes the resulting patch-and-joint tokens with a causal Transformer. This preserves local temporal structure while reducing the sequence length seen by attention.

Torque adaptor. The torque adaptor network in Figure 2 predicts the residual torque at the control rate from a short local window, the current nominal torque, and the latent state. It receives $(q, \dot{q}, \tau^a)$ over the last L control steps, with $L \ll K$, together with the current nominal torque τ_t^0 that should be corrected. For each joint, separate MLPs embed the local position, velocity, and torque-window features before they are combined with the nominal torque and latent state z . The per-joint latent state $z_{t,j}$ conditions the hidden layers through adaptive layer normalization [18, 19], allowing the same torque adaptor weights to express different correction laws under different inferred dynamics. The torque adaptor then predicts an additive residual torque for each joint,

$$\Delta\tau_t = A_\theta(q_{t-L:t}, \dot{q}_{t-L:t}, \tau_{t-L:t-1}^a, \tau_t^0, z_t). \quad (2)$$

This residual is added to the nominal torque to form the corrected torque τ_t^c , which is applied to the real or perturbed robot.

4 Experimental Results

We evaluate TAM on tasks where the policy is learned for an ideal robot but execution changes when the target robot has different dynamics. TAM training follows a two-stage simulation pipeline. First, we pretrain the full TAM module, including the history encoder and torque adaptor, on randomized rollouts pooled from five robot models (Panda, Piper, RBY-1, iiwa14, and Flexiv Rizon4). This multi-robot pretraining is used to learn a shared torque-correction initialization that can transfer across robot morphologies and serve as the starting point for robot-specific fine-tuning. Second, when target-robot simulation is available, we fine-tune the same TAM weights on that robot’s randomized rollouts. No policy weights are trained in either stage. All Panda-specific results below use the same Panda-specific fine-tuned TAM weights unless a row is explicitly labeled as multi-robot, robot-wise simulation fine-tuned, or an ablation. Across those Panda experiments, we change only the policy, its action space or low-level controller, and the target physical condition. **We evaluate three hypotheses:** (i) TAM improves motion alignment under unknown physical parameters, low-level-controller changes, and unseen robot models, (ii) the same weights can be attached below policies with different implementations, action spaces, and low-level controllers, and (iii) the history encoder, physics residual feature, and torque adaptor are important design choices. We present the most task-level real-world results first, then analyze motion alignment and cross-robot simulation behavior; ablations are reported in Appendix F.1. Table D.1 summarizes the experimental structure.

TAM receives only the nominal torque and robot-side history; it does not read policy observations, rewards, task labels, or low-level-controller internals. No task-specific fine-tuning, policy-specific adaptation, real-world calibration data, or pre-task activation motion is used. Unless a condition is explicitly labeled as carried-state, adaptation is reset at the start of each trial. TAM executes the nominal torque directly until the history encoder produces its first latent state, typically after ≈ 0.5 s, and this cold-start interval is included in the reported metric.

Figure 3 shows the real-world settings. We compare against Direct, which sends the nominal torque without adaptation, and Online SysID, which estimates explicit physical parameters online with a particle filter and analytically computes the corrected torque. PhysPred replaces the particle filter

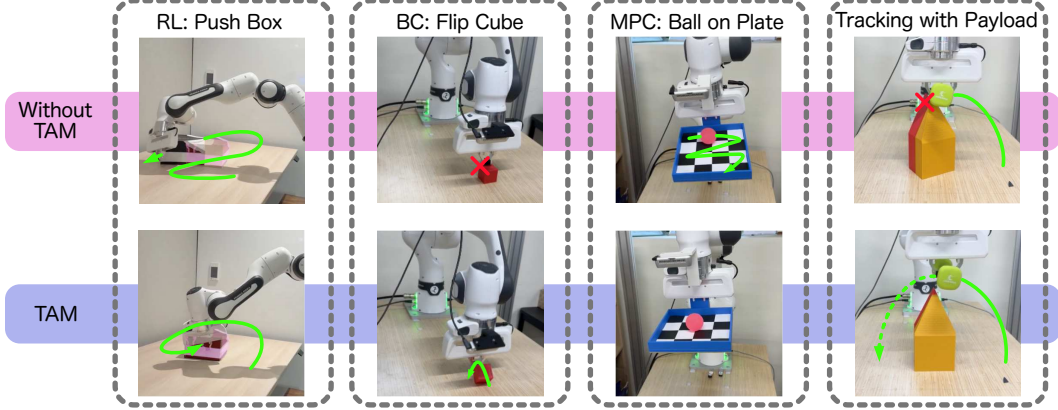


Figure 3: Representative real-world evaluations with direct execution (top, w/o TAM) and TAM-enabled execution (bottom). Columns show the four Panda settings used in the main experiments: RL box pushing, BC cube flipping, an MPC ball-on-plate policy, and tracking with an added payload.

Task	Metric	Reference	Real robot				
		Ideal sim	Direct	Online SysID	RMA	TAM (w/ RMA)	TAM
Box pushing (depth distill.)	Success rate (21 trials)	84.0%	47.6%	57.1%	52.4%	66.7%	76.2%
Behavior-cloned flip (RGB distill.)	Success rate (50 trials)	73.5%	50.0%	34.0%	—	—	72.0%
Ball-on-plate MPC	Goals / 15 s (20 runs)	15.9 ± 2.3	8.8 ± 2.4	13.1 ± 2.3	—	—	13.6 ± 2.9

Table 1: Real-world task statistics for multiple policies. Ideal sim reports the same policy evaluated in its ideal simulation, serving as an idealized upper-bound reference. Trial counts and time limits are given in the metric column; parenthetical labels indicate deployed visual modality. Online SysID is detailed in Appendix C, and RMA variants are only defined for RL box pushing.

with a learned history-to-parameter predictor; for RL pushing, RMA adapts inside the policy rather than at the torque interface. Baseline details are in Appendix C and Appendix E.1.

4.1 Multiple Policies and Action Spaces

Table 1 consolidates Direct, Online SysID, RMA, and TAM below the RL, BC, and MPC policies in Table D.1; dashes mark policy-side baselines that are not defined for a task. Across the three tasks, TAM outperforms all non-ideal baselines.

Vision-based pushing policies. We adopt the non-prehensile manipulation setup from MuJoCo Playground [20]. The task goal is to push the box to the commanded planar goal pose within the time limit. The pushing policy is trained in simulation as a vision policy for the RL pushing task; it consumes a partial point cloud, robot joint position and velocity, and the goal pose. Appendix E gives the policy-training, deployment, and success-threshold details. TAM improves over Direct and the strongest non-TAM baseline, Online SysID, by 28.6 and 19.1 percentage points, respectively, and also improves the RMA policy from 52.4% to 66.7%. Additional RMA diagnostics are in Appendix E.1.

Behavior-cloned flip policy. The task goal is to execute a contact-rich flip motion that brings the object to the desired final configuration. The flip policy is an ACT policy [21] trained from demonstrations generated by a scripted demonstrator in ideal-model simulation. At deployment, the policy receives a gripper-camera image and robot state, outputs joint targets, and a joint-impedance controller converts those targets into the nominal torque corrected by TAM. Appendix E gives the demonstration-generation and ACT-training details. This task stresses adaptation because the episode is short, contact timing is sensitive, and the decisive forceful interaction mixes external contact with robot-side mismatch. As shown in Table 1, TAM reaches 72% success, close to the 73.5% ideal-simulation baseline. This improves over direct transfer by 22 percentage points and over Online SysID by 38 percentage points, more than doubling its success rate. Online SysID performs worse than Direct, likely because the short contact-rich policy execution provides poor data for identifying robot parameters: the observed motion reflects both robot mismatch and object interaction.

Method	Real Panda (hidden params)		Real Panda + 1 kg payload	
	Reset	Carried	Reset	Carried
Direct	4.09 $\pm$ 0.39		6.04 $\pm$ 0.51	
Online SysID	2.67 $\pm$ 0.63	2.75 $\pm$ 0.58	4.69 $\pm$ 0.42	4.65 $\pm$ 0.53
PhysPred	1.74 $\pm$ 0.50	1.64 $\pm$ 0.56	4.15 $\pm$ 0.41	4.39 $\pm$ 0.35
TAM (ours)	1.51 $\pm$ 0.30	1.04 $\pm$ 0.24	2.35 $\pm$ 0.39	1.29 $\pm$ 0.35

Table 2: End-effector target-position tracking under two real-world physical conditions. All entries are target-position RMSE against the ideal-model reference in centimeters. “Real Panda (hidden params)” denotes the physical Franka Panda whose true physical parameters are not provided to the adaptation methods, while “Real Panda + 1 kg payload” intentionally adds an unknown 1 kg end-effector payload. For adaptive methods, Reset and Carried denote whether the online adaptation state is cleared or preserved at the start of the OSC segment; Direct has no online state and spans both columns. Values are mean $\pm$ standard deviation over 10 real-world trials per condition. Lower is better.

MPC ball-on-plate. The task goal is to repeatedly move the ball to commanded target positions on the plate within a 15 s time limit. The ball-on-plate MPC policy uses visual feedback and the ideal model to plan plate motions that move the ball toward target positions with an iLQR planner implemented through MuJoCo MPC [22]. This setting tests whether the same robot-side torque adaptor can support a model-based policy rather than a learned policy. Appendix E gives the estimator, MPC, and goal-counting details. Direct execution completes 8.8 ± 2.4 goals, Online SysID completes 13.1 ± 2.3 goals, and TAM completes 13.6 ± 2.9 goals, outperforming both baselines. The ideal-simulation baseline, which represents the policy’s performance with no sim-to-real gap, is 15.9 ± 2.3 goals per run.

4.2 Tracking: Motion Alignment Under Unknown Parameters and Low-Level Controller Switches

The tracking experiments isolate motion alignment. We evaluate real-world end-effector tracking on a Franka Emika Panda after switching from joint impedance to OSC under two physical conditions: the nominal robot and the same robot with an intentionally added 1 kg end-effector payload. This test asks whether TAM can make the target robot follow the ideal-model motion more closely when physical parameters or low-level controllers change. The TAM weights were trained on randomized joint-space waypoint rollouts and did not see any OSC-controlled trajectories during training.

Each trial is split into two segments and uses a sampled source trajectory: for each joint, the sine amplitude and period are randomized to generate a 16 s joint-space reference tracked by joint impedance control. At $t = 16$ s, the source controller is replaced by OSC, which tracks an end-effector pose trajectory for the remaining 8 s of the trial. Table 2 evaluates tracking error during this 8 s OSC segment. At the OSC switch, *Reset* clears adaptation state, while *Carried* preserves state inferred during the joint-impedance source segment. Appendix D.1 gives the full protocol, additional metrics, and representative carried-state payload trajectories in Figure D.1.

TAM gives the lowest tracking error in all four settings. Carried TAM reduces direct OSC error from 4.09 to 1.04 cm on the Panda instance and from 6.04 to 1.29 cm with the payload, while remaining below the explicit-parameter baselines. The reset and carried rows show that TAM can adapt from the target segment alone, but history collected under joint impedance further improves the later OSC segment.

4.3 Simulation Cross-Robot Generalization

We next ask whether TAM can reduce tracking error on robot models beyond the single robot used for the main real-world study. We use the same randomized tracking evaluation structure across robots, replacing the robot morphology while keeping the low-level-controller and evaluation protocol fixed. For each test robot, we instantiate its MuJoCo XML asset from MuJoCo Menagerie [23] and evaluate the multi-robot pretrained weights and robot-wise target-robot simulation fine-tuning. Appendix D.2 details the robot instantiation, DoF handling, and cross-robot e^T ablation. Table 3 reports simulated joint-position tracking RMSE in degrees.

The multi-robot TAM weights reduce tracking error on every reported robot, including the test-only robots, showing that torque-level correction can transfer across simulated robot models. Simulation

Robot	DoF	Split	Direct	Online SysID	TAM	TAM sim-ft.
Panda	7	Train	4.16 ± 0.99	1.43 ± 0.53	0.59 ± 0.19	0.43 ± 0.17
Piper	6	Train	4.88 ± 0.96	0.83 ± 0.18	<u>0.52 ± 0.17</u>	0.42 ± 0.15
iiwa14	7	Train	3.69 ± 0.82	0.99 ± 0.35	<u>0.56 ± 0.16</u>	0.47 ± 0.16
Flexiv Rizon4	7	Train	4.06 ± 1.20	1.33 ± 0.51	<u>0.84 ± 0.33</u>	0.68 ± 0.27
RBV-1	7	Train	3.80 ± 0.81	1.39 ± 0.83	<u>0.84 ± 0.16</u>	0.66 ± 0.17
Google Robot	7	Test	4.69 ± 1.13	1.65 ± 0.37	<u>1.05 ± 0.40</u>	0.93 ± 0.32
Unitree Z1	6	Test	6.11 ± 1.27	1.05 ± 0.23	<u>0.93 ± 0.19</u>	0.72 ± 0.24
Mean over robots	–	–	4.48	1.24	<u>0.76</u>	0.62

Table 3: Cross-robot tracking transfer in simulation. Values are joint-position RMSE in degrees over 100 randomized trials; lower is better. Split marks train/test robots, and “TAM sim-ft.” is robot-wise simulation fine-tuning from the multi-robot pretrained weights. Bold/underline mark best/second-best.

fine-tuning gives the lowest mean error across robots (0.62°), improving over multi-robot training (0.76°) and Online SysID (1.24°), while the test-only rows still show useful cross-robot structure without target-robot training. We therefore interpret multi-robot transfer as promising evidence of reusable correction structure, and simulation fine-tuning as the preferred practical path for obtaining the best target-robot performance. Appendix F.1 reports simulation ablations that isolate the history encoder, physics residual feature, and torque adaptor design choices.

5 Limitations

TAM has several scope limitations. It assumes access to a torque interface, a usable ideal inverse torque-command map, and matched kinematic structure and actuator ordering between the ideal and target robot. Robots with only position-level APIs or substantially different morphology may require a command-inversion layer or a different torque adaptor design. The training target is also local: TAM learns the torque needed for one-step motion alignment, not a formal guarantee of long-horizon stability, constraint satisfaction, or task success. All corrections are clipped to the robot’s jointwise torque budget, so payloads or disturbances that require torque beyond hardware authority remain outside scope. Contact-rich manipulation remains a limitation: randomized external torques bias TAM to preserve external effects, but deployment contacts, object dynamics, or force patterns outside this training distribution may still be confused with robot-side mismatch.

6 Conclusion

We presented **TAM**, a trajectory-conditioned adaptation layer that operates at the torque interface below a policy authored with respect to an ideal robot model: it compresses long-horizon proprioceptive and applied torque history into a compact latent state and adds a residual torque, so the perturbed plant’s local motion response matches the ideal robot’s response. With this design, a single shared set of TAM weights enables more robust sim-to-real transfer of learned policies that were not themselves trained with domain randomization, such as a vision-based RL box-pushing policy, a behavior-cloned flip policy, and an MPC ball-on-plate controller. Our cross-robot tracking study further shows that a single set of TAM weights generalizes across distinct manipulator embodiments, including robots never seen during training, and accuracy can be improved further by fine-tuning on simulated trajectories, requiring no real-world data collection. Overall, we have shown that TAM enables robust, dynamic manipulation¹.

¹Code, the trained TAM weights for the Franka Panda, and the simulation training distribution will be released.

References

- [1] Z. Xie, X. Da, M. van de Panne, B. Babich, and A. Garg. Dynamics randomization revisited: A case study for quadrupedal locomotion. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 4955–4961, 2021. doi:10.1109/ICRA48506.2021.9560837.
- [2] C. Gaz, M. Cognetti, A. Oliva, P. R. Giordano, and A. De Luca. Dynamic identification of the Franka Emika Panda robot with retrieval of feasible parameters using penalty-based optimization. *IEEE Robotics and Automation Letters*, 4(4):4147–4154, 2019. doi:10.1109/LRA.2019.2931248.
- [3] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 1–8, 2018. doi:10.1109/ICRA.2018.8460528.
- [4] H. W. Stone, A. C. Sanderson, and C. P. Neuman. Arm signature identification. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 41–48, April 1986.
- [5] S. Farsoni, C. T. Landi, F. Ferraguti, C. Secchi, and M. Bonfè. Real-time identification of robot payload using a multirate quaternion-based kalman filter and recursive total least-squares. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 2103–2109, 2018. doi:10.1109/ICRA.2018.8461167.
- [6] J. Swevers, C. Ganseman, D. B. Tukel, J. De Schutter, and H. Van Brussel. Optimal robot excitation and identification. *IEEE Transactions on Robotics and Automation*, 13(5):730–740, 1997. doi:10.1109/70.631234.
- [7] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26):eaau5872, 2019. doi:10.1126/scirobotics.aau5872.
- [8] X. Zhang, S. Liu, P. Huang, W. J. Han, Y. Lyu, M. Xu, and D. Zhao. Dynamics as prompts: In-context learning for sim-to-real system identifications. In *MARW at AAAI 2025*, 2025. URL <https://openreview.net/forum?id=8A41YAGe4u>.
- [9] F. Ramos, R. Possas, and D. Fox. BayesSim: Adaptive domain randomization via probabilistic inference for robotics simulators. In *Proceedings of Robotics: Science and Systems*, Freiburg im Breisgau, Germany, June 2019. doi:10.15607/RSS.2019.XV.029.
- [10] OpenAI, M. Andrychowicz, B. Baker, M. Chociej, R. Józefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, J. Schneider, S. Sidor, J. Tobin, P. Welinder, L. Weng, and W. Zaremba. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020. doi:10.1177/0278364919887447.
- [11] F. Muratore, F. Ramos, G. Turk, W. Yu, M. Gienger, and J. Peters. Robot learning from randomized simulations: A review. *Frontiers in Robotics and AI*, 9:799893, 2022. doi:10.3389/frobt.2022.799893.
- [12] W. Yu, J. Tan, C. K. Liu, and G. Turk. Preparing for the unknown: Learning a universal policy with online system identification. In *Proceedings of Robotics: Science and Systems*, Cambridge, Massachusetts, July 2017. doi:10.15607/RSS.2017.XIII.048.
- [13] A. Kumar, Z. Fu, D. Pathak, and J. Malik. RMA: Rapid motor adaptation for legged robots. In *Proceedings of Robotics: Science and Systems*, Virtual, July 2021. doi:10.15607/RSS.2021.XVII.011.

- [14] Z. Fu, X. Cheng, and D. Pathak. Deep whole-body control: Learning a unified policy for manipulation and locomotion. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2022. doi:10.48550/arXiv.2210.10044. URL <https://arxiv.org/abs/2210.10044>.
- [15] Y. Liang, K. Ellis, and J. F. Henriques. Rapid motor adaptation for robotic manipulator arms. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16404–16413, June 2024. doi:10.1109/CVPR52733.2024.01552.
- [16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.
- [17] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=Jbdc0vT0col>.
- [18] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018. URL <https://arxiv.org/abs/1709.07871>.
- [19] W. Peebles and S. Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4195–4205, 2023. doi:10.1109/ICCV51070.2023.00387.
- [20] K. Zakka, B. Tabanpour, Q. Liao, M. Haiderbhai, S. Holt, J. Y. Luo, A. Allshire, E. Frey, K. Sreenath, L. A. Kahrs, C. Sferrazza, Y. Tassa, and P. Abbeel. MuJoCo Playground. In *Robotics: Science and Systems (RSS)*, 2025. URL <https://playground.mujoco.org/>.
- [21] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. doi:10.15607/RSS.2023.XIX.016.
- [22] T. Howell, N. Gileadi, S. Tunyasuvunakool, K. Zakka, T. Erez, and Y. Tassa. Predictive sampling: Real-time behaviour synthesis with MuJoCo. *arXiv preprint arXiv:2212.00541*, 2022. doi:10.48550/arXiv.2212.00541. URL <https://arxiv.org/abs/2212.00541>.
- [23] K. Zakka, Y. Tassa, and MuJoCo Menagerie Contributors. MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo, 2022. URL https://github.com/google-deepmind/mujoco_menagerie.